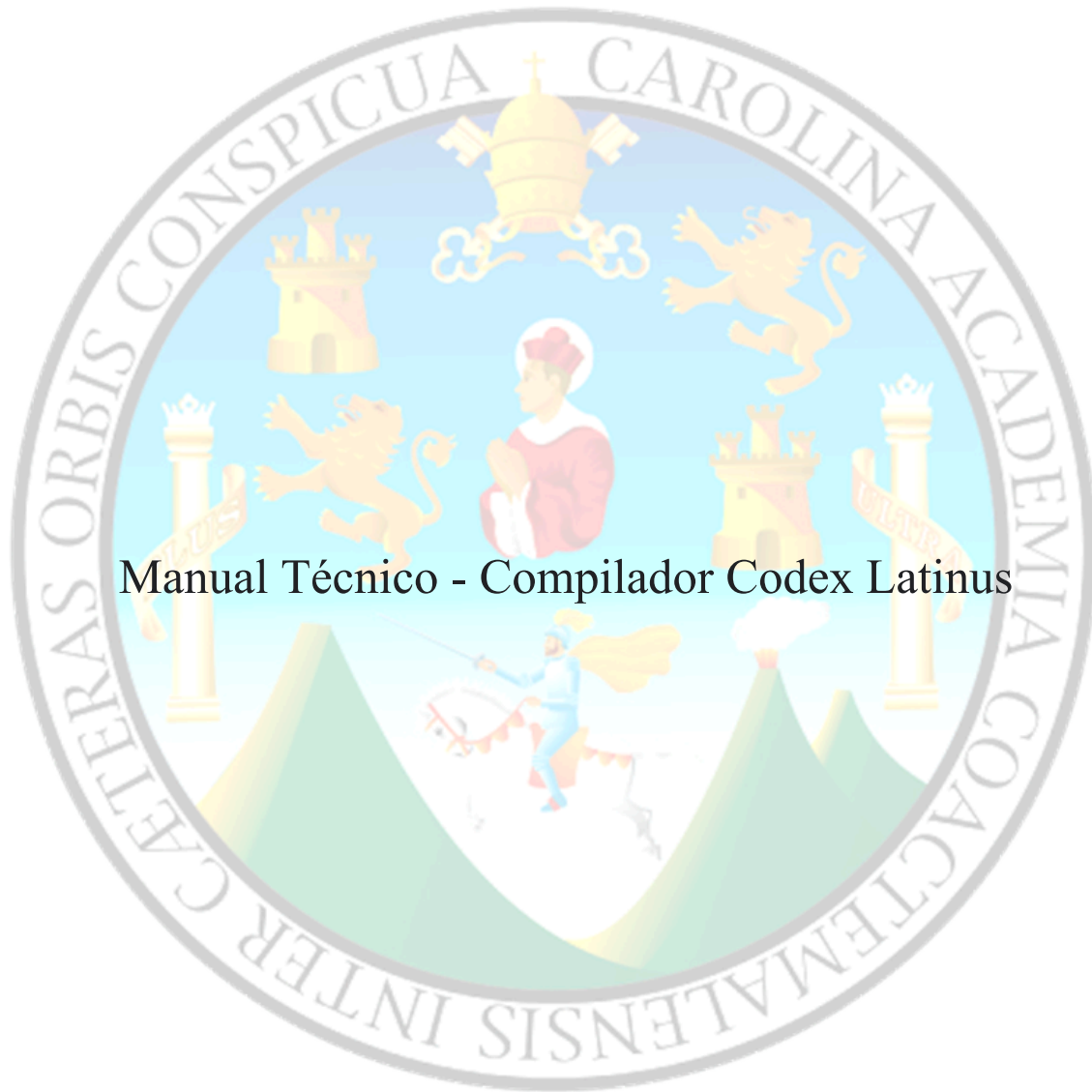




Manual Técnico - Compilador Codex Latinus

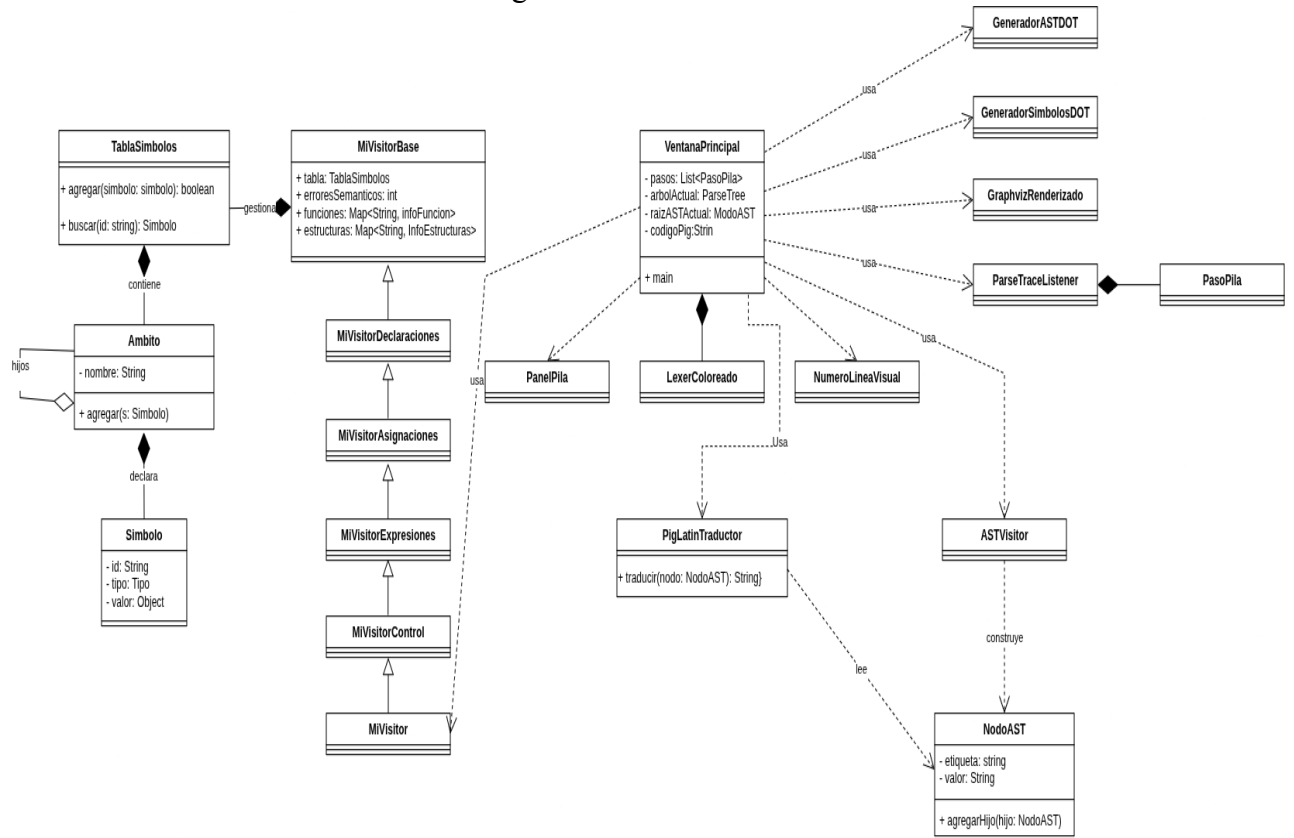
Mauricio Joel Gómez Barrios - 202031478

24 de agosto de 2026

Tecnologías Utilizadas.

- Java: Lenguaje de programación utilizado para el desarrollo de toda la lógica del compilador.
- ANTLR4: Generador de analizadores Lexer y Parser. A partir del archivo de gramática CodexLatinus.g4, ANTLR genera el código en Java para reconocer la sintaxis del lenguaje y construir automáticamente el Parse Tree. Se usó Visitor para recorrer el árbol.
- Java Swing: Se usó para desarrollar la Ventana Principal del usuario. muestra los componentes de la UI como el editor de texto, la salida de consola, pestañas y botones para realizar acciones.
- Graphviz: Se usó para la visualización gráfica externa desde la aplicación. A través del dot, el compilador transforma el código generado de los grafos en imágenes PNG y SVG para renderizar el AST y la Tabla de Símbolos.

Diagrama de Clases.



Especificaciones de Palabras Reservadas, Símbolos y Gramática.

- 1) Expresiones: Permiten evaluar valores y calcular resultados utilizando tipos de datos como: numerus, textum, decimalis y littera. Solo se permite operar valores primitivos directamente, los arreglos de datos (series) y las estructuras definidas por el usuario no se consideran primitivos en las operaciones aritméticas o lógicas pero los valores individuales que tienen en su interior sí son, por lo que pueden ser utilizados libremente en expresiones y llamadas a funciones.

los componentes que se permiten en expresiones son:

- Literales y variables primitivas.
- Acceso a posiciones específicas de arreglos (numeros[0]).
- Acceso a atributos de estructuras (persona.edad).

El compilador reconoce y procesa la siguiente jerarquía de operadores y símbolos:

Operador	Símbolos/palabras	Descripción
Aritméticos	+, -, *, /, %	Estándar universal
Relacionales	==, !=, <, >, >=, <=	Estándar universal
Lógicos	&&	And y Or
Negación	non	Negación universal
Suma/Resta abreviada	++ --	Suma/Resta una unidad

- 2) Declaraciones: Todas las declaraciones de variables deben ir dentro de sus respectivas secciones habilitadas como VARIABLES> a nivel global o VARIABLES[...] a nivel local.

Ejemplo:

```
// Declaración de entero (numerus) con valor inicial
```

```
esto mi_entero : numerus 10;
```

```
esto total : numerus fuerza * 2;
```

```
// Declaración de texto (textum)
```

```
esto nombre : textum "Somos la resistencia";
```

```
// Declaración de decimal (decimalis)
```

```
esto gravedad : decimalis 9.81;
```

```
// Declaración de un carácter (littera)
```

```
esto inicial : littera 'a';
```

```
// Declaración de booleano simplificado
```

```
esto activo : bool verum;
```

- 3) Booleanos: Se representan mediante las palabras reservadas *verum* para verdadero y *falsus* para falso además de que se puede usar la palabra reservada *bool* de forma opcional y exclusiva para la definición rápida de variables booleanas simples (esto *mi_booleano : verum;*), aunque se prefiere mantener el tipado explícito general.
- 4) Arreglos (Series): Permiten almacenar colecciones de datos bajo una misma estructura. Los tipos soportados pueden ser de cualquier tipo primitivo (*numerus*, *textum*, *decimalis*, *bool*, *littera*) o de cualquier estructura definida por el usuario. Los arreglos comienzan su numeración en el índice 0 y es posible modificar o asignar valores a posiciones específicas utilizando expresiones como índices. El índice de un arreglo puede ser cualquier expresión numérica (*numerus*). Si dicha expresión puede evaluarse durante el análisis semántico, el compilador validará que no pase los límites y lanzará un error en caso de salir del rango declarado.

Ejemplo:

```
// Array de enteros con valores iniciales
series mis_enteros[2] : numerus {1, 1};
// Array de enteros sin inicialización (tamaño reservado)
series mis_enteros_[2] : numerus;
// Array de texto con valores iniciales
series nombres[2] : textum {"Hola", "Adios"};
series nombres_[2] : textum;
nombres[0] = "Capitán Espárragos";
nombres[1] = nombres[0] + " clon";
```

- 5) Asignaciones e Inferencia de Tipos: Para garantizar la integridad de las transmisiones y evitar errores lógicos, el compilador usa un análisis semántico estricto de Verificación de Tipos y un sistema de inferencia implícita basado en una jerarquía de niveles:
1. *textum* (nivel más alto: 5)
 2. *decimalis* (Nivel: 4)
 3. *numerus* (Nivel: 3)
 4. *littera* (Nivel: 2)
 5. *bool* (*verum* / *falsus*) (Nivel más bajo: 1)

Las reglas de compatibilidad y operación son:

- Al operar un *numerus* con un *decimalis*, el resultado se promueve automáticamente al tipo de mayor jerarquía (*decimalis*).
- Sumar un *textum* con un *littera* da como resultado un *textum* mediante concatenación.
- *textum* es el único tipo de dato incompatible con las operaciones aritméticas (-, *, /, %). Solo permite la operación de suma (+) para concatenar con otros tipos.
- Cualquier intento de mezclar tipos no compatibles sin una relación válida será reportado como error semántico.

Ejemplo:

```
// Asignación numérica directa
esto mi_entero : numerus 10;
mi_entero = 23;
```

```
// Asignación y evaluación lógica booleana
esto mi_booleano : falsus;
mi_booleano = verum || 1 == 1;
// Concatenación e inferencia con cadenas
esto mi_cadena : textum "Nothing here";
mi_cadena = "Abajo el imperio porcino " + 100 + " no tenemos miedo";
// Operación aritmética con promoción a decimal
esto mi_float : decimalis 12.4;
mi_float = 17 / 2
```

- 6) Uso de Estructuras: Las estructuras permiten al usuario definir tipos de datos compuestos personalizados. Cuentan con una sección y sintaxis específica donde se verifica obligatoriamente que no existan nombres de atributos duplicados:

Ejemplo:

```
structura Persona {
    esto nombre: textum;
    esto edad: numerus;
} finis;
```

Para declarar una variable de tipo estructura, se especifican sus valores de forma explícita (el orden de los atributos no altera la validación):

Ejemplo:

```
esto mi_personaje : Persona {
    nombre: "Yennifer",
    edad: 999
}
```

El lenguaje soporta la composición de estructuras complejas que combinan tipos primitivos, arreglos y otras estructuras:

Ejemplo:

```
structura Animal {
    esto nombre: textum,
    esto apodo: textum
} finis;
```

```
structura Selva {
    esto valido : verum,
    series animales : Animal
} finis;
```

- 7) Estructuras de control:

- Condicionales (si / aliter): La condición de evaluación debe ser estrictamente booleana. Evaluar un valor numérico o incompatible es detectado como un error de flujo y se marca como error semántico. Soporta bloques simples, división binaria (si / aliter), y múltiples validaciones encadenadas con condiciones intermedias (aliter (expresion)).

- Ciclos:
 1. dum (Mientras): Ejecución iterativa condicionada previamente.
 2. facere ... dum (Hacer-Mientras): Garantiza la ejecución de las instrucciones del bloque al menos una vez antes de evaluar la condición.
 3. per (Para): Bucle controlado por un iterador con declaración, condición y actualización en una sola línea (per (esto i : numerus 0; i < 10; i++)).
 4. Interrupciones: Las palabras reservadas perge (continuar) e interrompe (romper) se pueden utilizar exclusivamente dentro de los ciclos para alterar el flujo de ejecución.

8) Uso de Funciones: Permite modularizar el código fuente en rutinas independientes para evitar patrones repetitivos frente al análisis del tráfico.

Tipo de funciones:

1. actio: Funciones de ejecución sin valor de retorno (void).
2. ratio [tipo]: Funciones que devuelven un valor tipado específico.

Las variables locales de una función deben declararse estrictamente al principio de la misma dentro del bloque VARIABLES[...], y nunca después de estructuras de control.

El analizador semántico valida rigurosamente que:

1. El valor retornado mediante reddere coincide con el tipo esperado.
2. Existe una ruta de retorno obligatoria en todos los caminos posibles de la función.
3. No exista código inalcanzable ("no rastreable") posterior a una instrucción de retorno.

9) Funciones Especiales y Estructura del Programa:

Funciones del Sistema:

Lectura (<<): Captura texto desde la consola. Puede utilizarse de forma aislada (<<) o asignarse directamente a una variable de tipo textum (mi_textum <<).

Impresión (>>): Envía salidas a la consola. Permite encadenar múltiples impresiones consecutivas de forma ilimitada (>> "Hola" >> variable;).

Ejemplo completo:

```
##
  Sección opcional de variables, puede no existir
  En esta sección solo se definen variables, arreglos
  o estructuras globales
##
VARIABLES>
esto edad : numerus 20;
esto cifrado : falsus;
esto comandante : textum "Estudiante X";
esto fuerza : numerus 10;
esto poder : numerus 0;

##
  Sección de funciones, es opcional
```

En esta seccion solo se deben definir funciones

##

MUNERA>

// Definición de funcion con retorno

ratio numerus calcularPoder(esto fuerza : numerus) {

VARIABLES[

esto total : numerus fuerza * 2;

]

reddere total;

} finis;

##

Seccion de funcion principal

Esta sección es obligatoria

##

MAIOR>

>> "Hola comandante!" ;

>> "Ingresa tu nombre por favor" ;

comandante <<

>> "Bienvenido" >> comandante ;

>> "Ingresa tu edad" ;

edad <<

si (edad >= 18) {

cifrado = verum;

fuerza = 12;

} finis ;

>> "Tu poder es: >> " calcularPoder(fuerza);

>> "La puerta esta cifrada?" >> cifrado ;

FINIS;

Gráfica de la Pila Llamadas.

Para ver el proceso de análisis sintáctico y permitir análisis sobre la validación de la estructura del lenguaje, el compilador implementa un sistema de visualización paso a paso de la Pila de Llamadas del analizador (ANTLR4).

Se construyó la clase ParseTraceListener (heredando de CodexLatinusBaseListener), la cual se acopla al ParseTreeWalker. Este listener intercepta los eventos en tiempo real durante la construcción del árbol:

- Push: Al ingresar a una regla gramatical (enterEveryRule), el nombre de la regla se apila.
- Pop: Al salir de la regla (exitEveryRule), esta se desapila.
- Shift: Al reconocer un token terminal (visitTerminal), se registra la acción de consumo del token.
- Error: Registra recuperaciones o fallos léxicos/sintácticos (visitErrorNode).

El estado de la pila y el historial de acciones se encapsulan en objetos PasoPila. La clase PanelPila expone esta información mediante una interfaz controlada con botones de navegación direccional (Anterior, Siguiente, Reiniciar, Ir al final). La vista divide la pantalla mostrando la pila de reglas activas invertida (para simular el tope de la pila visualmente) y un log textual (JTextArea) con la traza exacta de operaciones que llevaron a dicho estado.

Traducción a Pig Latin mediante AST.

Para evadir las restricciones en las comunicaciones de la Resistencia, el compilador incluye la traducción de código hacia el lenguaje Pig Latin. Esta traducción se realiza mediante un recorrido estructural y semántico.

La traducción es gestionada por la clase PigLatinTraductor, la cual recibe la raíz del árbol (NodoAST) validado previamente por el analizador semántico. El traductor evalúa cada nodo recursivamente y reconstruye el código fuente aplicando las reglas únicamente sobre identificadores, palabras reservadas y tipos de datos, manteniendo intactos los símbolos estructurales y de operación. Implementado en el método traducirPalabra(). Si el motor detecta que un identificador o palabra reservada inicia con una vocal (A, E, I, O, U), la palabra se conserva intacta y se le concatena el sufijo way al final (actio a actioway).

Si el identificador inicia con una o más consonantes consecutivas, el algoritmo extrae dicho bloque de consonantes, lo desplaza al final de la palabra y le añade el sufijo ay (structura a ucturastray, fuerza a uerzafay).

Al visitar los nodos correspondientes a las funciones de sistema, el traductor intercepta las operaciones y las transforma a sus equivalentes nativos en Pig Latin:

Las instrucciones de impresión (originalmente >>) se renderizan como %OINK.

Las instrucciones de lectura (originalmente <<) se renderizan como %OINK_OINK.

Esta arquitectura garantiza que la traducción muestre la estructura lógica del programa original sin corromper el alcance de los bloques, permitiendo la generación de un archivo de salida .pig perfectamente válido y descargable desde la interfaz de la Ventana Principal.

Tabla de Compatibilidad de Tipos.

Operación	Izquierdo	Operador	Derecho	Resultado	Comportamiento del Compilador
Concatenación	textum	+	<i>primitivo</i>	textum	El primitivo se convierte implícitamente a texto y se une a la cadena original.
Concatenación	<i>primitivo</i>	+	textum	textum	El primitivo se convierte implícitamente a texto y se une a la cadena original.
Aritmética Mixta	decimalis	+, -, *, /, %	decimalis/ numerus	decimalis	Sube automáticamente al tipo de mayor jerarquía (Nivel 4).
Aritmética Mixta	numerus	+, -, *, /, %	decimalis	decimalis	Sube automáticamente al tipo de mayor jerarquía (Nivel 4).
Aritmética Pura	numerus	+, -, *, /, %	numerus	numerus	Operación entre enteros, el resultado se mantiene en su nivel (Nivel 3).
Restricción Lógica	textum, littera, bool	-, *, /, %	<i>Cualquier tipo</i>	ERROR SEMÁNTICO	Estos tipos no soportan matemáticas.
Asignación Mismo Tipo	<i>primitivo</i>	=	<i>Mismo primitivo</i>	VÁLIDO	Asignación directa en memoria sin necesidad de conversiones.
Asignación Ascendente	<i>Tipo de Nivel Mayor</i>	=	<i>Tipo de Nivel Menor</i>	VÁLIDO	Promoción (un decimalis puede recibir un numerus sin problema).
Asignación Descendente	<i>Tipo de Nivel Menor</i>	=	<i>Tipo de Nivel Mayor</i>	ERROR SEMÁNTICO	Incompatible por pérdida de precisión (un numerus no puede recibir un decimalis).